

Week 4: Media Query, HTML 5, and JavaScript Beginning

Column Properties | Writing Mode

Column Properties & Writing Mode allow text content to be displayed in multiple columns and control the flow direction of text. **column-count** splits content into columns, while **column-gap** and **column-rule** manage spacing and dividers. **writing-mode** defines text direction, such as horizontal or vertical flow.

Key Properties:

- **column-count**: Number of columns
- **column-gap**: Space between columns
- **column-rule**: Border between columns
- **writing-mode**: Sets text flow direction (**horizontal-tb**, **vertical-rl**, etc.)

Ex.

```
.text {  
  column-count: 2;  
  column-gap: 30px;  
  writing-mode: vertical-rl;  
}
```

Multiple Background Images

Multiple Background Images in CSS allow you to layer several background images on a single element. They are defined using the **background-image** property, separated by commas. Each image can have its own position, size, and repeat settings, enabling creative visual designs like overlays or textures.

Ex.

```
.box {  
  background-image: url('top.png'), url('bottom.png');  
  background-position: top left, bottom right;  
  background-repeat: no-repeat;  
}
```

Background Gradient

Background Gradient creates a smooth transition between two or more colors as a

background. It can be linear or radial, and is defined using **linear-gradient()** or **radial-gradient()** functions to enhance visual appeal without using images.

Ex.

```
.box {  
  background: linear-gradient(to right, red, orange, yellow);  
}
```

max-height

max-height sets the maximum height an element can grow to. If the content exceeds this limit, it overflows or scrolls based on **overflow** settings, ensuring layout consistency.

Ex.

```
.box {  
  max-height: 400px;  
  overflow-y: auto;  
}
```

min-height

min-height defines the minimum height an element must maintain, even if its content is smaller. It ensures that the element doesn't shrink below a certain height, helping maintain layout stability and design consistency.

Ex.

```
.box {  
  min-height: 200px;  
}
```

max-width

max-width sets the maximum width an element can expand to, preventing it from becoming wider than the specified value. It's useful for responsive designs to ensure content doesn't stretch too far on large screens.

Ex.

```
.container {  
  max-width: 1200px;  
  margin: auto;  
}
```

min-width

min-width sets the minimum width an element must maintain, even if its content is smaller. It ensures that the element doesn't shrink below a specified width, helping preserve layout structure and readability.

Ex.

```
.sidebar {  
  min-width: 300px;  
}
```

object-fit

Defines how an image or video should be resized to fit its container. It works on replaced elements like `<img>` or `<video>`.

Ex.

```
img {  
  object-fit: cover; /* Fills container, crops overflow */  
}
```

object-position

Sets the position of an image or video inside its container when using **object-fit**. Helps control the visible area of cropped media.

Ex.

```
img {  
  object-fit: cover;  
  object-position: top center; /* Aligns image from the top */  
}
```

filter

Applies visual effects like blur, brightness, grayscale, and contrast to elements (especially images).

Ex.

```
img {  
  filter: grayscale(100%); /* Converts image to black & white */  
}
```

image-rendering

Controls the scaling behavior of images. It affects how images are smoothed or pixelated when resized, useful for pixel art or crisp graphics.

Ex.

```
img {  
  image-rendering: pixelated; /* Makes images appear blocky when scaled */  
}
```

Introduction to Media Query

Media Queries in **CSS3** allow you to apply different styles to a webpage based on the device's screen size, resolution, or type (like mobile, tablet, desktop). They are essential for creating responsive designs.

1. What is a media query?

A media query is a feature of CSS3 that allows you to apply CSS styles based on the device's characteristics, such as:

- Screen width and height
- Device type (screen, print, etc.)
- Orientation (portrait or landscape)

It helps make websites responsive, meaning they adjust their layout and design to look good on all screen sizes (mobile, tablet, desktop).

Example: -

```
@media (max-width: 600px) {  
  
  body {  
  
    background-color: lightgray;  
  
  }  
}
```

Meaning: If the screen width is **600px or less**, the background color will change to **light gray**.

2. How does it work?

Media queries check the characteristics of the user's device (like screen width, height, orientation, etc.) and apply specific CSS styles only if the conditions are true.

- Browser reads the screen/device properties.
- It checks if the media query condition is true.
- If the condition matches, the CSS inside the media query is applied.
- If not, those styles are ignored.

Example: -

```
@media (max-width: 768px) {  
  .menu{  
    display: none;  
  }  
}
```

Meaning: If the screen is **768 pixels** wide or smaller, the .menu element will not be displayed (**hidden**). On larger screens, it will show normally.

3. Mobile first approach v/s Desktop first approach

Feature	Mobile-First Approach	Desktop-First Approach
Definition	Design for small screens first (like mobiles), then scale up.	Design for large screens first (like desktops), then scale down.
CSS Media Queries	Uses min-width to apply styles for larger screens.	Uses max-width to apply styles for smaller screens.
Performance	Faster on mobile (loads fewer resources first).	May load heavy content first, then hide it for mobile.
Best For	Responsive, mobile-friendly websites.	Projects focused on desktop with minor mobile support.

Example Media Query	@media (min-width: 768px) { ... }	@media (max-width: 768px) { ... }
Preferred Today?	Yes, widely recommended for modern web design.	Less common unless the desktop audience dominates.

HTML5 | Introduction

HTML5 is the latest version of the HyperText Markup Language used to create and structure web content. It introduced new semantic elements (`<header>`, `<footer>`, `<article>`), multimedia support (`<audio>`, `<video>`), form enhancements, and APIs for better performance, mobile compatibility, and interactivity in modern web development.

HTML5 Semantic and Non-Semantic Elements

Semantic elements clearly describe their meaning (e.g., `<header>`, `<article>`, `<footer>`), improving SEO and accessibility. Non-semantic elements (like `<div>`, `<span>`) don't convey meaning and are used for styling or scripting.

Ex.

```
<article>This is a semantic element.</article>
<div>This is non-semantic.</div>
```

Basic HTML5 Structure

The basic HTML5 structure includes a `<!DOCTYPE html>` declaration, and elements like `<html>`, `<head>`, and `<body>`. This structure ensures compatibility with modern browsers.

Ex.

```
<!DOCTYPE html>
<html>
  <head>
    <title>My HTML5 Page</title>
  </head>
  <body>
    <h1>Hello HTML5!</h1>
  </body>
</html>
```

HTML5 Tags: Semantic Elements

- **<header>** – Defines the page or section header.
- **<footer>** – Defines the footer for a section or page.
- **<aside>** – Represents side content like a sidebar.
- **<figure>** – Encapsulates media or images with a caption.
- **<figcaption>** – Provides a caption for **<figure>**.
- **<article>** – Represents independent content like blog posts.
- **<section>** – Defines a section within a page.
- **<main>** – Highlights the main content area.

Ex.

```
<main>
<article>
  <header><h2>News</h2></header>
  <section>Today's top stories...</section>
  <footer>Posted on June 11</footer>
</article>
</main>
```

HTML5 Other Tags

- **** – Emphasizes text (italicized by default).
- **<mark>** – Highlights text like a marker.
- **<small>** – Displays small-sized text.
- **** – Indicates deleted content (strikethrough).
- **<ins>** – Represents inserted content (underlined).
- **<sub>** – Subscript text.
- **<sup>** – Superscript text.

Ex.

```
<p>This is <em>important</em> and <mark>highlighted</mark>.</p>
<p>Price: <del>$100</del> <ins>$80</ins></p>
<p>H<sub>2</sub>O and x<sup>2</sup></p>
```

HTML5 Tags

1. **<video>** – Embeds video files.

Ex.

```
<video controls width="300">
  <source src="movie.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

2. **<audio>** – Embeds audio files.

Ex.

```
<audio controls>  
  <source src="audio.mp3" type="audio/mpeg">  
  Your browser does not support the audio tag.  
</audio>
```

3. **<iframe>** – Embeds external webpages.

Ex.

```
<iframe src="https://example.com" width="300" height="200"></iframe>
```

4. **<details>** – Creates a collapsible section.
5. **<summary>** – Defines the visible heading for **<details>**.

Ex.

```
<details>  
  <summary>Click to see more</summary>  
  <p>This is hidden content shown when expanded.</p>  
</details>
```

Introduction to GIT and GitHub

GIT is a version control system that helps track changes in code and collaborate efficiently. GitHub is a cloud-based platform that hosts Git repositories and allows multiple developers to work together on the same project. It enhances collaboration through features like pull requests, issues, and version history.

Ex.

```
git init  
git add .  
git commit -m "Initial commit"  
git push origin main
```

See GIT in Action

Using Git in real-time involves creating repositories, tracking changes, branching for new features, and merging code. Developers can revert, review, or share code updates across teams, making workflows efficient and safe.

Ex.

```
git clone https://github.com/user/repo.git
git checkout -b feature-login
git commit -am "Add login form"
git push origin feature-login
```

Must-Know Things for Developers

Essential Git skills include using commands like **add**, **commit**, **push**, **pull**, **clone**, and **branch**. Understanding version history, conflict resolution, and staging is crucial. Developers should also know how to write meaningful commit messages and manage **.gitignore**.

Ex.

```
git status
git log
echo "node_modules/" > .gitignore
```

Using GIT for Collaboration

Git enables collaboration by allowing developers to work on different branches and merge them safely. GitHub adds tools like pull requests, code reviews, and issue tracking to streamline teamwork and ensure quality contributions.

Ex.

```
# Create a new branch
git checkout -b bugfix/header-issue
```

```
# Push and collaborate via GitHub
git push origin bugfix/header-issue
```

Beginning with JavaScript

JavaScript is a powerful, lightweight programming language used to make web pages interactive. It's a case sensitive language. It runs in the browser and allows you to create dynamic features like sliders, popups, form validation, animations, and more. Javascript is not Java.

1. Why Do We Use JavaScript?

JavaScript is used to make websites dynamic, interactive, and user-friendly. It runs in the browser and can respond to user actions without reloading the page.

- Make Web Pages Interactive
- Form Validation
- Dynamic Content Update
- Animations & Effects
- Browser-Based Games or Apps
- Fetch Data Asynchronously (AJAX)

Example:

```
<p id="demo">Hello!</p>  
<button onclick="document.getElementById('demo').innerText = 'Welcome to  
JavaScript!'">Click Me</button>
```

Meaning: Changes the paragraph text when the button is clicked.

2. How to Use JavaScript?

You can use **JavaScript** in three main ways in an HTML page:

- **Inline JavaScript**

JavaScript is written **directly in an HTML tag** using the onclick, onmouseover, etc. attributes.

Ex. `<button onclick="alert('Hello!')">Click Me</button>`

- **Internal JavaScript**

JavaScript is written **inside a <script> tag** within the HTML file (usually in the <head> or at the end of <body>).

Ex.

```
<!DOCTYPE html>  
<html>  
  <head>  
    <script>  
      function greet() {  
        alert("Welcome!");  
      }  
    </script>  
  </head>  
  <body>  
    <button onclick="greet()">Click Me</button>  
  </body>  
</html>
```

- **External JavaScript**

JavaScript code is written in a **separate .js file**, then linked using the `<script src="...">` tag.

Ex.

```
script.js
function greet() {
  alert("Hello from external file!");
}
```

```
index.html
<script src="script.js"></script>
<button onclick="greet()">Click Me</button>
```

3. Variables in JavaScript

- **Var**

- a. Declares a variable (older method).
- b. **Function-scoped** and can be **re-declared & updated**.
- c. Not recommended in modern JavaScript.

Ex.

```
var name = "John";
name = "Mike";
var name = "David";
```

- **let**

- a. Declares a **block-scoped** variable.
- b. Can be **updated** but **not re-declared** in the same block.
- c. Preferred for variables that may change.

Ex.

```
let age = 25;
age = 30;
```

- **Const**

- a. Declares a **block-scoped** constant.
- b. Cannot be **updated or re-declared**.
- c. Use for values that should never change.

Ex.

```
const PI = 3.14;
```

Conditional Statement in JavaScript

Conditional statements are used to make decisions in JavaScript. They execute different blocks of code based on whether a condition is true or false.

- **If Statement**

Runs a block of code only if the condition is true.

Ex.

```
let score = 90;  
if (score > 80) {  
  console.log("Excellent!");  
}
```

Output :- Excellent

- **if...else Statement**

Runs one block if the condition is true, and another if it's false.

Ex.

```
let age = 16;  
if (age >= 18) {  
  console.log("You can vote.");  
} else {  
  console.log("You are too young to vote.");  
}
```

Output :- You are too young to vote.

- **if...else if...else**

Check multiple conditions in order.

Ex.

```
let marks = 75;
```

```
if (marks >= 90) {  
  console.log("Grade A");  
} else if (marks >= 70) {  
  console.log("Grade B");  
} else {  
  console.log("Grade C");  
}
```

Output :- Grade B